Project - High Level Design

On

Marketing Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Devi Modi	EN23CS3T1008
2.	Bhoomika Sanodiya	EN22CS301271
3.	Bhoomika Chauhan	EN22CS301275
4.	Dollyna Sahu	EN22CS301354
5.	Bhavya Rathore	EN22CS301269

Group Name:Group02-D3

Project Number:AAI-26

Industry Mentor Name:

University Mentor Name: Prof. Maya Baniya Yadav

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

This document presents Marketing Planning Assistant Agent is an Agentic AI system built to automate the process of creating a complete marketing campaign plan from a single high-level goal. Traditional marketing planning requires a team of specialists — a researcher, a strategist, a scheduler, and a report writer working over several days to produce a campaign document. This project replaces that manual process with five AI agents that execute sequentially and deliver a structured marketing plan in under five minutes.

The project is built using CrewAI as the multi-agent orchestration framework, Groq as the free-tier LLM provider running the llama-3.1-8b-instant model, Python 3.12 as the programming language, and Streamlit for the web interface.

1.1. Scope of the document

This document defines the detailed low-level design of all agent, task, tool, and utility classes , Sequential data flow from user input through five AI agents to final output ,User flow of the Streamlit web application ,Entity definitions and relationships between system components ,Technology stack selection .

1.2. Intended Audience

This document is intended for:

- Developers implementing the system
- Academic evaluators and mentors
- Students studying Agentic AI
- Researchers exploring AI in Marketing Planning Assistant Agents

It helps the audience understand system design, workflows, and technical implementation details.

1.3. System overview

The system is a multi-agent AI-based Marketing Planning assistant that processes user-defined goals and generates structured marketing plans.

Figure 1: Use Case Diagram

Working Overview

1. User provides a Marketing goal
2. Planner Agent interprets the goal
3. Goal is decomposed into smaller tasks
4. Specialized agents perform tasks
5. Tools validate data and resources
6. Final execution plan and report is generated

Key Features

- Multi-agent collaboration
- Tool-based validation
- Dynamic planning and scheduling

2. System Design

The system follows a modular multi-agent architecture where each component performs a specific role. The design ensures flexibility, scalability, and efficient task execution.

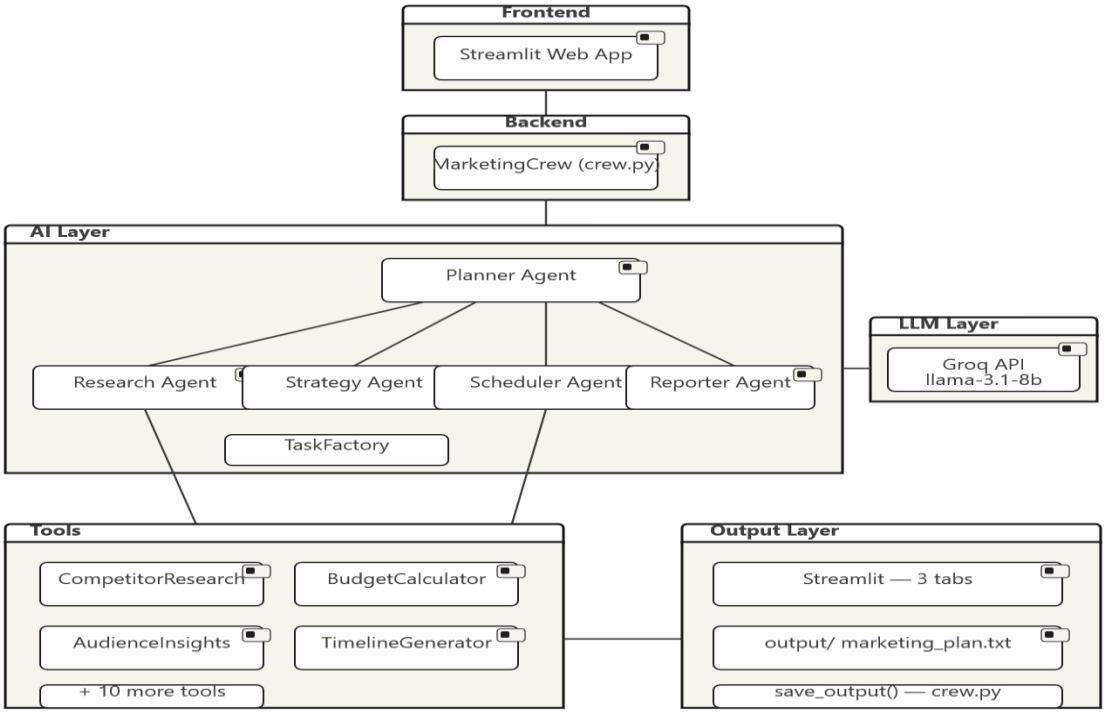

Figure 2: Architecture Diagram

Architecture Flow

User Input → Planner Agent → Task Decomposition → Agents → Tools → Output

2.1. Application Design

The application consists of the following modules:

1. Input Module

- Accepts marketing goal from user in natural language
- Validates that the goal is non-empty and meaningful
- Enriches input with additional parameters — budget, duration, and target market

- Passes the final goal string to the Planner Agent

2. Planner Agent

- Core decision-making unit
- Breaks goals into smaller tasks using LLM

3. Agent Layer

- Research Agent analyzes the market, identifies 3 competitors with their messaging and weaknesses, builds 2 audience personas, and finds 5 relevant market trends
- Strategy Agent builds the campaign strategy including positioning statement, channel mix, budget allocation across channels, and 7 measurable KPIs with targets and deadlines
- Scheduler Agent creates the 12-week execution timeline divided into 4 phases — Pre-Launch, Launch, Optimize, and Evaluate — with task owners and dependencies
- Reporter Agent aggregates all previous agent outputs and writes the complete 8-section executive marketing plan report

4. Tool Layer

- Competitor Research Tool
- Audience Insights Tool
- Market Trends Tool
- Budget Calculator Tool
- Channel Selector Tool
- KPI Generator Tool
- Timeline Generator Tool
- Dependency Mapper Tool
- Resource Allocator Tool
- Report Formatter Tool
- Risk Assessor Tool

5. Output Module

- Aggregates results
- Generates structured Marketing plan

2.2. Process Flow

The system follows a structured execution flow:

1. User inputs marketing goal
2. System validates input
3. Planner Agent decomposes the goal
4. Tasks are assigned to agents
5. Agents execute tasks
6. Tools are used for validation
7. Results are combined
8. Final plan is generated

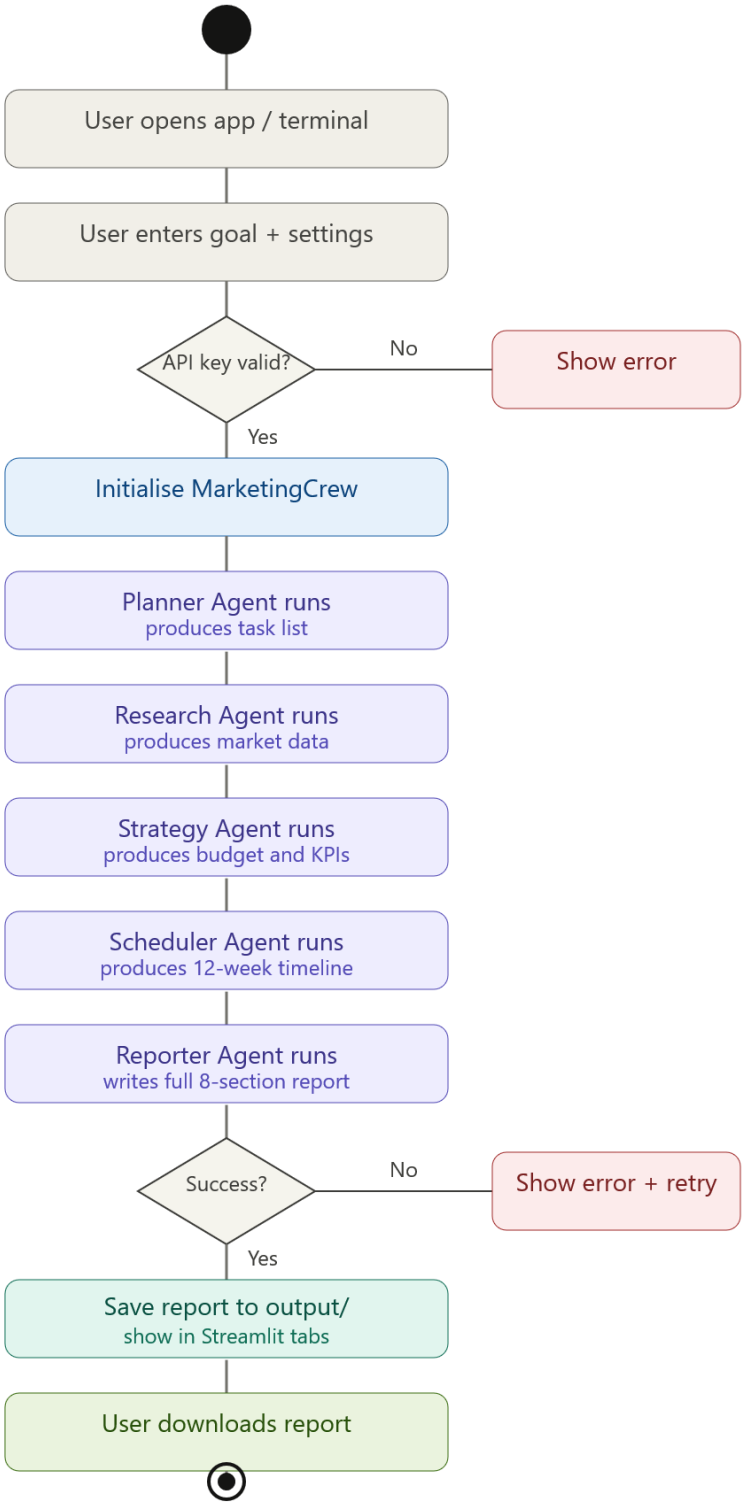

Figure 3: Activity Diagram

2.3 Information Flow

The system follows a sequential agent-based workflow, where data moves step-by-step from user input to final output. Each stage transforms the information, adding more context and refinement, resulting in a structured marketing plan. This flow ensures clarity, modular processing, and better control over each stage of execution.

Key Characteristics of Flow

- Sequential and controlled execution
- Context-aware (each step builds on previous)
- Modular and easy to extend
- Ensures structured and high-quality output

The flow of information in the system is as follows:

User Input
↓
Planner Agent
↓
Task List
↓
Agents
↓
Tool Calls
↓
Processed Data
↓
Final Output

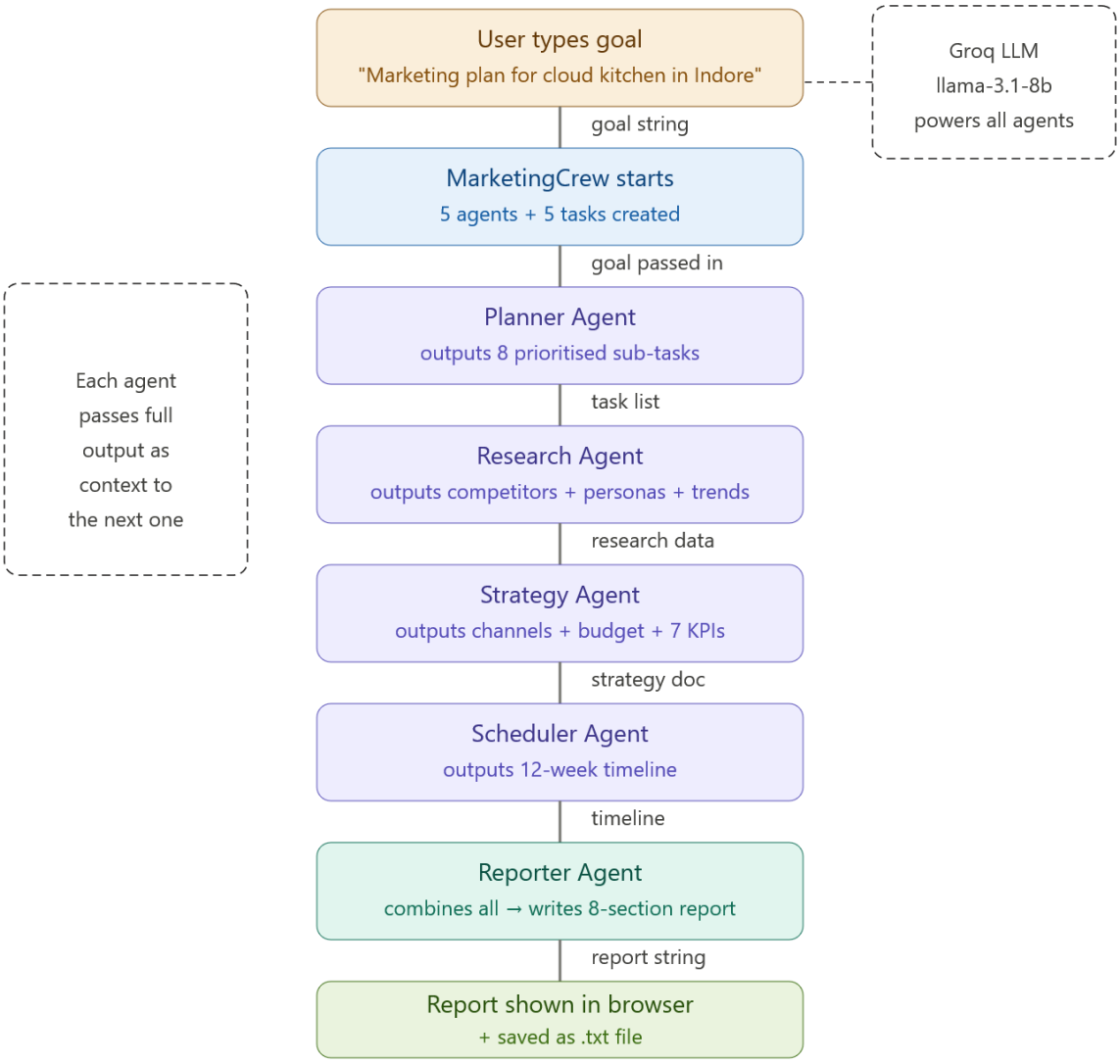

Figure 4: Flow Chart

Data Handling

- Input is processed and structured
- Intermediate data is shared between agents
- Final output is formatted for readability

2.4. Components Design

1. Planner Agent

- Responsible for task decomposition
- Uses LLM for reasoning and planning

2. Research agent

- Analyzes patient symptoms
- Identifies possible conditions

3. Strategy Agent

- Builds the full campaign strategy from research findings
- Selects the top 3 to 4 marketing channels with budget justification

4. Scheduler Agent

- Converts strategy into a 12-week execution timeline
- Divides work into 4 phases — Pre-Launch (Weeks 1–3), Launch (Weeks 4–6), Optimize (Weeks 7–10), Evaluate (Weeks 11–12)
- Assigns owner roles to each task and identifies critical path dependencies

5. Reporter Agent

- Synthesizes all previous agent outputs into a polished executive document
- Writes 8 sections — Executive Summary, Goal, Research, Strategy, Schedule, Risks, Quick Wins, Appendix
- Adds risk assessment with top 3 risks and mitigations

6. LLM Integration

- Uses Groq API
- Model: llama3-8b-8192
- Temperature = 0.3

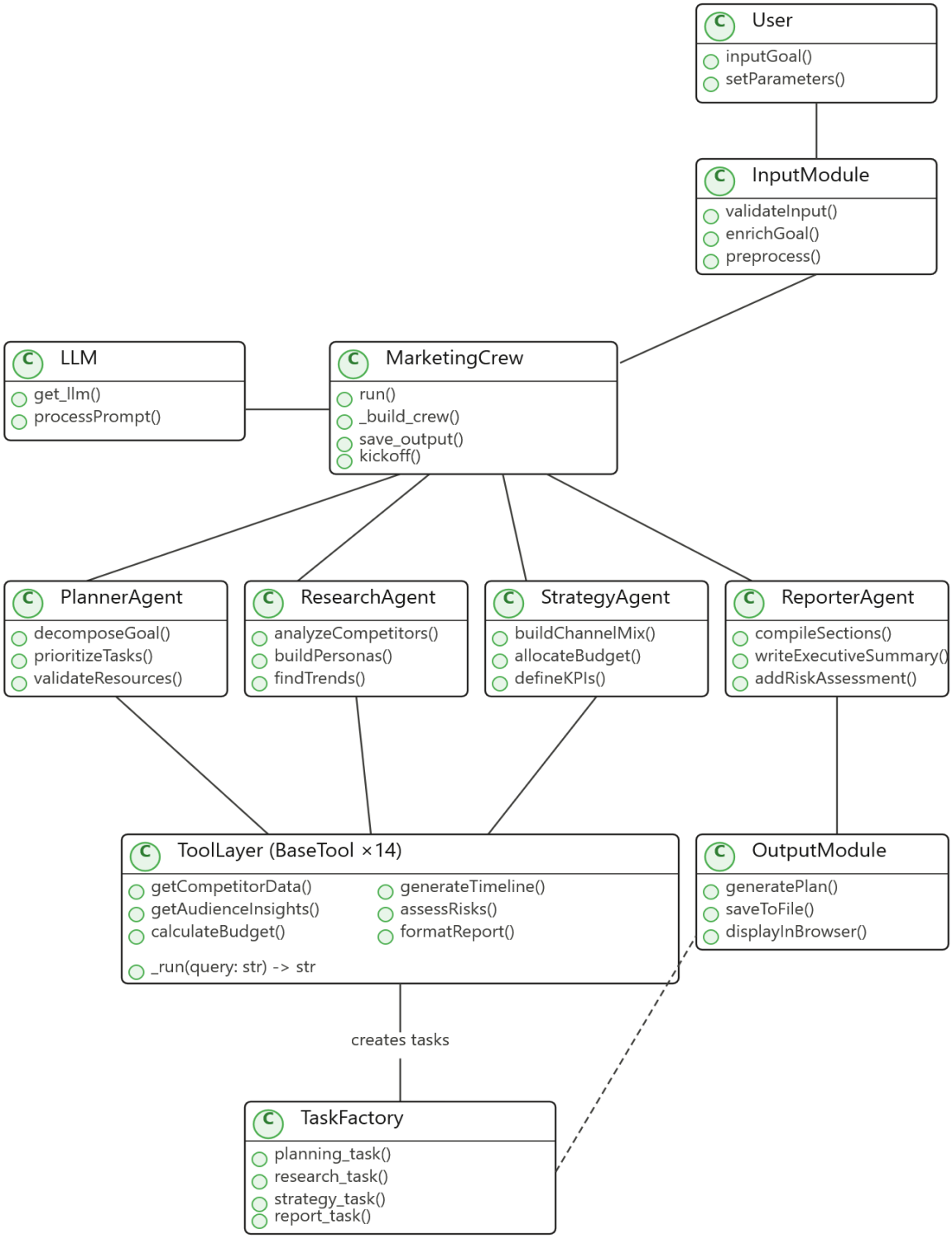

Figure 5: Class Diagram

2.5 Key Design Considerations

The system is designed with a focus on simplicity, scalability, and modular architecture. It ensures that components can evolve independently while maintaining overall system efficiency. These considerations help in making the system easy to extend, maintain, and deploy, especially when transitioning from prototype to production.

Scalability

- Supports multiple agents working in sequence
- Can be extended to handle parallel execution in production
- Scales with better backend (e.g., FastAPI + queue systems)

Modularity

- Each agent and component is independent
- Easy to update or replace individual modules
- Reduces impact of changes on the overall system

Flexibility

- New agents and tools can be added easily
- Supports different marketing use cases
- Configurable inputs and workflows

Performance

- Uses efficient LLM models for faster response
- Background execution keeps UI responsive
- Optimized for prototype-level speed

Reliability

- Mock tools reduce dependency on external APIs
- Ensures stable and predictable outputs
- Minimizes failure points during execution

Security

- No real or sensitive user data is used
- Local execution ensures data privacy
- Minimal external data exposure

2.6 API Catalogue

The system integrates both **external and mock APIs** to perform various tasks such as content generation, research, and planning. These APIs provide structured inputs and outputs, enabling smooth communication between system components.

1. LLM API (Groq)

Used for generating responses and executing agent tasks using a large language model.

Details:

- **API Key** → Used for authentication
- **Model** → llama3-8b-8192
- **Input** → Prompt (user goal + context)
- **Output** → Generated response text

2. Competitor Research API (Mock)

Simulates competitor analysis for a given industry or market.

Input:

- Industry or market keyword

Output:

- Competitor profiles including:
  - Ad spend
  - Marketing channels
  - Messaging strategy
  - Weaknesses

3. Audience Insight API (Mock)

Provides insights about the target audience based on the product or campaign.

Input:

- Product description or target market

Output:

- Audience insights such as:
  - Demographics
  - Preferences
  - Behavior patterns
  - Pain points

4. Budget Calculator API (Mock)

Helps in distributing the campaign budget across different channels.

Input:

- Campaign goal type
- Total budget

Output:

- Channel-wise budget allocation:
  - Percentage split
  - Suggested spending per channel
  - Basic reasoning for allocation

3. Data Design

3.1 Data Model

The Marketing Planning Assistant Agent is a **stateless, single-run system**. It does not use any database, and all data exists only during execution. Once the session ends, all in-memory data is lost except the final report saved as a text file.

Main Data Objects:

- **User Input**
  - Marketing goal (string)
  - Campaign parameters: budget, duration, target market
  - Timestamp
  - Combined into a single enriched input before processing
- **Agent Output**
  - One output per agent (5 agents total)
  - Contains:
    - Agent name & index
    - Generated response
    - Completion timestamp
  - Stored temporarily in a Python dictionary
- **Final Report**
  - Includes:
    - Original goal
    - Run timestamp
    - Combined outputs of all agents
  - Saved as a .txt file (only persistent data)
- **Task Object (CrewAI)**
  - Holds:
    - Prompt description
    - Expected output
    - Context (previous agent outputs)
  - Exists only during execution

3.2 Data Retention Policies

The system follows a **minimal data retention approach**, suitable for a prototype with no sensitive data. It is designed to ensure **privacy, simplicity, and low storage overhead**, since no long-term user tracking or analytics is required. All processing is session-based, making the system lightweight and easy to manage.

Memory Data

- Stored only during runtime (RAM)
- Includes:
  - Agent outputs
  - Task objects
  - LLM responses
- Automatically cleared after session ends (no recovery)

Report Files

- Stored in output/ folder
- Features:
  - Saved as .txt
  - Unique timestamp-based filenames
  - No auto deletion (manual management required)

API Keys

- Stored in .env file locally
- Loaded at startup only
- Not logged or exposed externally (except API request)

No User Data Storage

- No:
  - Login system
  - Session history
  - Usage tracking
- Each run is completely independent

Production Recommendation

- Use:
  - PostgreSQL → store run metadata
  - ChromaDB → enable semantic search on reports

3.3 Data Migration

Since the current version uses no database, there is no data migration requirement for the prototype.However, when this system is upgraded to a production version with a database,

the following migration plan applies:

Phase 1 - Local to database migration All existing .txt report files in the output/ folder will be read, parsed, and inserted into a PostgreSQL reports table. Each file will produce one row with columns for goal, date, content, and file path.

Phase 2 - Vector embedding Each report will be passed through an embedding model and stored in ChromaDB. This enables semantic search — a user can search for past plans by describing a goal and retrieve the most similar previous plan.

Phase 3 - Schema versioning Future changes to the report structure will be managed using Alembic for PostgreSQL migrations. Each schema change will have an upgrade and downgrade script to ensure rollback capability.

Phase 4 - Multi-user migration When multi-user support is added, all existing single-user reports will be assigned to a default admin user account. New runs will be tagged with the authenticated user's ID.

4. Interfaces

The system provides three types of interfaces to ensure flexibility and ease of use for different users. It supports both graphical and command-line interaction, making it suitable for beginners as well as developers. The interface design focuses on simplicity, real-time feedback, and clear visualization of agent outputs during execution.

User Interface (Streamlit Web App)

The primary interface is a Streamlit app (localhost:8501) that allows users to input goals and view results interactively. It provides a user-friendly layout with real-time updates, making it easy to track the progress of each agent. The UI is designed to minimize complexity while still offering full visibility into the system workflow.

Key Features:

- Sidebar:
  - API key status
  - Agent roles overview
- Main Panel:
  - Goal selection (preset + custom)
  - Campaign settings (budget, duration, market)

- Execution:
  - Run button triggers pipeline
  - Real-time agent status (grey → yellow → green)
- Output:
  - Expandable agent outputs
  - Live logs with timestamps
  - Tabs for:
    - Final report
    - Outputs
    - Download
    - Summary dashboard

- CLI (Terminal)
- Run using: python main.py
- Menu-based input and console output

5. State and Session Management

The system is stateless, with all data stored temporarily during execution. It does not maintain any persistent session or user-specific state, which simplifies the architecture and reduces overhead. This design ensures that each execution is independent, isolated, and free from previous run interference, improving reliability.

State Variables (in memory):

- agent_outputs → stores outputs of all agents
- current_agent → tracks running agent
- final_result → Reporter Agent output
- full_report → compiled report
- error_msg → stores errors if any

Session Lifecycle

- Starts: when user clicks **Run**
- Ends: when app closes or page refreshes
- No:
  - Login
  - Session storage
  - Recovery mechanism

6. Caching & Security

The system currently implements no caching, meaning every request is processed fresh to ensure up-to-date results. This simplifies the architecture and avoids stale outputs but increases API usage and response time. Security is handled at a basic level, focusing on safe data transmission and minimal exposure, as the system is designed for local, single-user usage.

Caching

- No caching implemented
- Every run triggers fresh LLM API calls
- No reuse of previous results or responses

Security & Data Handling

Data Usage

- Only marketing goals are processed
- No personal, financial, or sensitive data collected

Data in Transit

- Communication with API uses **HTTPS (TLS encryption)**
- API key sent securely in request headers

Prompt Injection Risk

- Possible via custom user input
- Low risk in local environment
- Needs validation in production

Authentication

- No login or user authentication
- No role-based access control

Dependencies

- Uses secure open-source libraries:
  - CrewAI
  - LangChain
  - Streamlit
- Recommended to pin versions for stability

Production Recommendations

- Add:
  - Input validation & sanitization
  - Authentication (JWT / OAuth2)
  - HTTPS deployment
  - API rate limiting and monitoring

7. Performance Aspects

The system performance is mainly influenced by LLM response time and API rate limits. Although the underlying model is fast, the overall execution time is increased due to sequential agent processing and intentional delays. The system is optimized for clarity and reliability rather than speed, making it suitable for prototype usage.

Response Time

- Total runtime: **4–6 minutes**

- Breakdown:
  - Each agent: ~15–30 seconds
  - Delay between agents: ~25 seconds
- UI remains responsive due to background execution

Throughput

- Supports:
  - 1 user
  - 1 pipeline at a time
- Limited by API rate limits and architecture

Model Performance

- Uses LLaMA 3.1 8B Instant (Groq)
- Speed: ~500–800 tokens/sec
- Actual generation time is very low (~5 sec per agent)
- Delay mainly due to rate limiting

Scalability Limitations

- Streamlit is single-threaded
- Sequential agent execution increases total time
- Shared API limits across runs

Production Recommendations

- Use:
  - FastAPI + React (scalable architecture)
  - Celery + Redis (background task queue)
- Upgrade API plan for higher limits
- Run agents in parallel (reduce runtime to ~2 minutes)

8. References

- CrewAI Documentation
- LangChain Documentation
- Groq API Documentation
- Research papers on Agentic AI
- Marketing Planning Assistant AI system design resources